

DeepLearning_HW14_Python solution

● Filtering with a threshold on class scores

```
def yolo_filter_boxes(box_confidence, boxes, box_class_probs, threshold = .6):
    # Step 1: Compute box scores
    ### START CODE HERE ### (~ 1 line)
    box_scores = box_confidence*box_class_probs
    ### END CODE HERE ###

    # Step 2: Find the box_classes thanks to the max box_scores, keep track of the
    ### START CODE HERE ### (~ 2 lines)
    box_classes = K.argmax(box_scores, axis = -1)
    box_class_scores = K.max(box_scores , axis=-1)
    ### END CODE HERE ###

    # Step 3: Create a filtering mask based on "box_class_scores" by using "threshold"
    # same dimension as box_class_scores, and be True for the boxes you want to keep
    ### START CODE HERE ### (~ 1 line)
    filtering_mask = box_class_scores >= threshold
    ### END CODE HERE ###

    # Step 4: Apply the mask to scores, boxes and classes
    ### START CODE HERE ### (~ 3 lines)
    scores = tf.boolean_mask(box_class_scores, filtering_mask)
    boxes = tf.boolean_mask(boxes,filtering_mask)
    classes = tf.boolean_mask(box_classes,filtering_mask)
    ### END CODE HERE ###

    return scores, boxes, classes
```

● IOU

```
def iou(box1, box2):
    """Implement the intersection over union (IoU) between box1 and box2
    Arguments:
    box1 -- first box, list object with coordinates (x1, y1, x2, y2)
    box2 -- second box, list object with coordinates (x1, y1, x2, y2)
    """

    # Calculate the (y1, x1, y2, x2) coordinates of the intersection of box1 and box2
    ### START CODE HERE ### (~ 5 lines)
    xi1 = max(box1[0],box2[0])
    yi1 = max(box1[1],box2[1])
    xi2 = min(box1[2],box2[2])
    yi2 = min(box1[3],box2[3])
    inter_area = (yi2-yi1)*(xi2-xi1)
    ### END CODE HERE ###

    # Calculate the Union area by using Formula: Union(A,B) = A + B - Inter(A,B)
    ### START CODE HERE ### (~ 3 lines)
    box1_area = (box1[3]-box1[1])*(box1[2]-box1[0])
    box2_area = (box2[3]-box2[1])*(box2[2]-box2[0])
    union_area = box1_area + box2_area - inter_area
    ### END CODE HERE ###

    # compute the IoU
    ### START CODE HERE ### (~ 1 line)
    iou = inter_area/union_area
    ### END CODE HERE ###

    return iou
```

● Non-max suppression

```
def yolo_non_max_suppression(scores, boxes, classes, max_boxes = 10, iou_threshold = 0.5):  
  
    max_boxes_tensor = K.variable(max_boxes, dtype='int32') # tensor to be used in tf.image  
    K.get_session().run(tf.variables_initializer([max_boxes_tensor])) # initialize variable  
  
    # Use tf.image.non_max_suppression() to get the list of indices corresponding to boxes  
    ### START CODE HERE ### (~ 1 Line)  
    nms_indices = tf.image.non_max_suppression(boxes, scores, max_boxes, iou_threshold=0.5)  
    ### END CODE HERE ###  
  
    # Use K.gather() to select only nms_indices from scores, boxes and classes  
    ### START CODE HERE ### (~ 3 Lines)  
    scores = K.gather(scores, nms_indices)  
    boxes = K.gather(boxes, nms_indices)  
    classes = K.gather(classes, nms_indices)  
    ### END CODE HERE ###  
  
    return scores, boxes, classes
```

● Yolo

```
def yolo_eval(yolo_outputs, image_shape = (720., 1280.), \  
             max_boxes=10, score_threshold=.6, iou_threshold=.5):  
  
    ### START CODE HERE ###  
  
    # Retrieve outputs of the YOLO model (~1 Line)  
    box_confidence, box_xy, box_wh, box_class_probs = yolo_outputs  
  
    # Convert boxes to be ready for filtering functions  
    boxes = yolo_boxes_to_corners(box_xy, box_wh)  
  
    # Use one of the functions you've implemented to perform Score-filtering with a thresh  
    scores, boxes, classes = yolo_filter_boxes(box_confidence, \  
                                              boxes, box_class_probs, score_threshold)  
  
    # Scale boxes back to original image shape.  
    boxes = scale_boxes(boxes, image_shape)  
  
    # Use one of the functions you've implemented to perform Non-max suppression with a th  
    scores, boxes, classes = yolo_non_max_suppression(scores, boxes, \  
                                                    classes, max_boxes, iou_threshold)  
  
    ### END CODE HERE ###  
  
    return scores, boxes, classes
```

● Loading a pre-trained model (YoLo)

1. 下載 yad2k.git

指令: git clone <https://github.com/allanzelener/yad2k.git>

2. 預先安裝的環境封包

指令:

```
pip install numpy
```

```
pip install h5py
```

```
pip install pillow
```

```
pip install tensorflow-gpu (GPU 版本)
```

```
conda install -c conda-forge tensorflow (CPU 版本)
```

```
pip install keras
```

3. 下載 Darknet model cfg 和 weights

指令: `git clone https://github.com/pjreddie/darknet`

複製目錄 [darknet/cfg/yolov2.cfg](#) 到目錄 [yad2k](#) 下

下載 weights

<https://pjreddie.com/darknet/yolov2/>

將 weight 文件複製到 [目錄 yad2k](#) 下

4. 將 Darknet YOLO 模型轉換為 Keras 模型 h5

至目錄 [yad2k](#) 下

指令: `cd yad2k`

執行將 YOLO 模型轉換為 Keras 模型

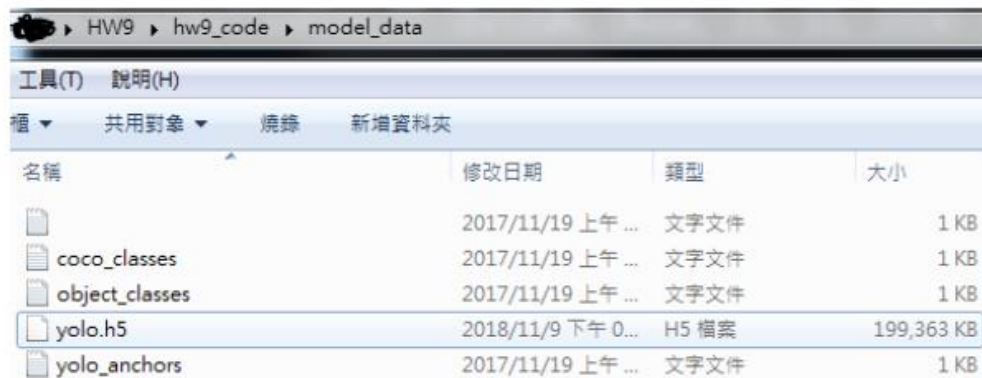
指令: `python ./yad2k.py yolov2.cfg yolov2.weights model_data/yolo.h5`

執行結果如下圖:

```
=====
Total params: 50,983,561
Trainable params: 50,962,889
Non-trainable params: 20,672

None
Saved Keras model to model_data/yolo.h5
```

5. 將 yolo.h5 移至 目錄 **model_data** 下如圖:



名稱	修改日期	類型	大小
coco_classes	2017/11/19 上午 ...	文字文件	1 KB
object_classes	2017/11/19 上午 ...	文字文件	1 KB
yolo.h5	2018/11/9 下午 0...	H5 檔案	199,363 KB
yolo_anchors	2017/11/19 上午 ...	文字文件	1 KB

接下來就可以使用 `load_model()` 引用 yolo.h5

安裝參考網站:

<https://github.com/allanzelener/YAD2K>

https://blog.csdn.net/weixin_41043240/article/details/79745614